Karadeniz Teknik Üniversitesi

Fen Fakültesi – Bilgisayar Bilimleri Bölümü

Hafta 11: Eşzamanlılık Desenleri ve OBYS

Yazılım Geliştirme II – Çalışma Kağıdı

Ad Soyad: _____	**Numara:** _____	**Tarih:** _____

Yönergeler

- Bu çalışma kağıdı **5 bölümden** oluşmaktadır.
- Çoktan seçmeli sorularda doğru seçeneği **daire içine alın**.
- Boşluk doldurma sorularında cevabınızı **çizgi üzerine** yazın.
- Doğru/yanlış sorularında **D** veya **Y** yazın.
- Süre: **40 dakika**
- Toplam: **60 puan**

Bölüm A: Çoktan Seçmeli Sorular

1. Bir thread-safe queue'da sadece `empty`, `front`, `pop` metodlarını ayrı ayrı kilitlemek neden yetersiz kalır? **2 puan**

   A) Bireysel metodlar yeterince hızlı değildir
   B) Kullanıcı bu üç metodu ardışık çağırırken arada başka thread durumu değiştirebilir; *arayüz yarışı* oluşur
   C) `std::queue` thread-safe bir sınıf olduğu için ek koruma anlamsızdır
   D) `lock_guard`, `front` metodu ile uyumsuzdur

2. std::shared_mutex hangi senaryoda std::mutex'ten kesin olarak daha yavaştır? **2 puan**

   A) Çoğunluk okuma, az yazma
   B) Çok kısa kritik bölge ve düşük thread sayısı (örneğin 2 thread, 50 ns'lik kilitli iş)
   C) İki thread'in nadir paralel okuma yaptığı senaryo
   D) Hiçbir zaman daha yavaş olmaz

3. Sharded ConcurrentMap'te bölmeler neden std::vector<Bolme> yerine std::deque<Bolme> (veya std::vector<std::u
   içinde saklanmalıdır? **2 puan**

   A) std::deque daha az bellek kullanır
   B) std::mutex taşınamaz/kopyalanamaz; vector büyürken iç dizisini taşırsa mutex'ler bozulur
   C) std::vector thread-safe değildir
   D) std::deque'in iteratörleri daha hızlıdır

4. C++20 std::jthread'in std::thread'e göre **en önemli avantajı** hangisidir? **2 puan**

   A) Daha hızlı bağlam değiştirme
   B) Daha az bellek kullanımı
   C) Otomatik join (destructor'da), `stop_token` ile iptal desteği ve istisna güvenliği
   D) Çoklu çekirdekleri otomatik kullanma

5. QProgressDialog'un `canceled` sinyali, worker'ın durdur slot'una bağlanırken neden `Qt::DirectConnection` tercih edilir? **2 puan**

A) Daha az bellek kullanır
B) Worker thread'ine sinyal hemen ulaşsın diye; aksi halde worker olay kuyruğunda bekleyebilir
C) Sinyal kuyruğunun bozulmasını engellemek için
D) Qt belgelerinde tek desteklenen bağlantı türü olduğu için

Bölüm B: Boşluk Doldurma

Aşağıdaki cümlelerde boş bırakılan yerleri uygun ifadelerle doldurun.

1. std::shared_lock<std::shared_mutex> okuyucular için kullanılır ve içeride _____shared çağırır; std::unique_lock<std::shared_mutex> ise yazıcılar için kullanılır ve içeride _____ çağırır. **2 puan**
2. C++20 ile gelen std::_____token bir thread'e iptal istendiğini bildirmek için kullanılır; ilgili kaynak std::_____source'tur. Thread içinde _____requested ile kontrol edilir. **2 puan**
3. Atomic sayaçla ilerleme bildirirken yazıcı tarafta std::memory_order_____, okuyucu tarafta std::memory_order_____ kullanmak "bitti" bayrağıyla sayacı senkronlamak için yeterlidir. **2 puan**
4. Toplu CSV içe aktarmada, condition_variable'in bekleyen tüketicilerini iptal anında uyandırmak için std::stop_____ kullanılır; bu sayede wait'in predicate'i stop_requested'i kontrol edebilir. **2 puan**
5. QProgressDialog'un setMinimumDuration(_____) metodu, kısa süren işlerde dialog'un **açılmamasını** sağlar; bu sayede UI'da _____ (titreme) önlenir. **2 puan**

Bölüm C: Doğru / Yanlış

Aşağıdaki ifadelerin doğru mu yanlış mı olduğunu belirtin.

1. std::shared_mutex, mevcut bir shared kilidi otomatik olarak unique kilide yükseltebilir; bu işlem standart kütüphane tarafından destek altındadır. **2 puan**
2. Bir worker thread'de `ilerleme` sinyali her satır için emit edilirse, UI olay kuyruğu dolar ve UI yavaşlar; bu yüzden 50-100 satırda bir emit yapılmalıdır. **2 puan**
3. std::jthread'in destructor'ı önce request_stop sonra join çağırır; thread süresi dolmamış olsa bile düzgün sonlanma garanti edilir. **2 puan**
4. Sharded ConcurrentMap'te aynı anda `a` ve `b` anahtarlı iki kayıt güncellemesi atomik olarak yapılabilir; bu yapı ACID garantisi sunar. **2 puan**
5. `Qt::QueuedConnection` ile bağlanmış bir sinyal, alıcı thread'in olay döngüsünde bir mesaja dönüşür ve slot orada çalıştırılır; bu sayede thread sınırı güvenli geçilir. **2 puan**

Bölüm D: Kod Okuma

1. Aşağıdaki kodda tryPop metodu hatalıdır. Sorunu açıklayın ve nasıl düzeltileceğini belirtin: **10 puan**

```cpp
template <typename T>
class TSQueue {
    std::queue<T> kuyruk_;
    std::mutex m_;
public:
    bool empty() {
        std::lock_guard<std::mutex> g(m_);
        return kuyruk_.empty();
    }
    T tryPop() {
        if (empty()) {                // (1)
            throw std::runtime_error("Bos");
        }
        std::lock_guard<std::mutex> g(m_);  // (2)
        T x = kuyruk_.front();
        kuyruk_.pop();
        return x;
    }
};
```

2. Aşağıdaki kodda neden std::deque<Bolme> kullanıldı, std::vector<Bolme> kullansaydık ne olurdu? Açıklayın: **10 puan**

```cpp
struct Bolme {
    std::mutex m;
    std::unordered_map<int, std::string> veri;
};

class CMap {
    std::deque<Bolme> bolmeler_;      // <-- vector olsa?
public:
    explicit CMap(std::size_t n) : bolmeler_(n) {}
    // koy(), al() ...
};
```

Bölüm E: Kod Yazma

1. std::jthread ve std::stop_token kullanarak, 0'dan 100'e kadar sayıları yazdıran ama her sayıdan önce iptal kontrolü yapan bir program yazın. Ana fonksiyon thread'i 30 ms sonra iptal etmelidir. **10 puan**

Başarılar!